



UNIVERSIDAD DON BOSCO

FACULTAD DE INGENIERIA

ESCUELA DE COMPUTACION

Guía de laboratorio N° 2

Ciclo II

Nombre de la práctica: Estructuras de control: Sentencias repetitivas.

Lugar de ejecución: Centro de cómputo.

Materia: Desarrollo de Aplicaciones Web con Software Interpretado en el Cliente.

I. Objetivos

- ☐ Dominar el uso de las sentencias repetitivas, ciclos o lazos del lenguaje JavaScript.
- ☐ Implementar sentencias repetitivas en la solución de problemas que requieran repetir un conjunto de instrucciones.
- ☐ Implementar sentencias repetitivas anidadas con lenguaje JavaScript.
- ☐ Utilizar sentencias de control de ciclos o lazos (*break* y *continue*).

II. Introducción Teórica.

Sentencias repetitivas.

Las sentencias repetitivas son el medio que brindan los lenguajes de programación para poder repetir un bloque o conjunto de instrucciones más de una vez. Estas sentencias suelen llamarse lazos, ciclos o bucles. El número de veces que se repite el ciclo o lazo es controlado mediante una condición o mediante el valor de un contador. Cuando se trata de una condición, se involucra una variable cuyo valor cambia cada vez que se ejecuta el lazo. En el caso de una variable contador aumenta de valor de forma automática, cada vez que se ejecuta el lazo hasta llegar a un valor máximo definido en el contador.

JavaScript proporciona varios tipos de sentencias repetitivas o lazos, entre ellos se pueden mencionar: *for*, *while* y *do-while*. Otras instrucciones particulares de JavaScript, relacionadas con el uso de objetos, son *for-in* y *with*.

Sentencia *for*

Permite crear un lazo o bucle que se ejecutará un número determinado de veces. Utiliza una variable contador, una condición de comparación, una instrucción de incremento (o decremento) del contador y una o más instrucciones que forman parte del cuerpo del lazo o bucle. Estas instrucciones se repetirán tantas veces hasta que la condición de comparación se evalúe como falsa.

La sintaxis de la sentencia *for* es la siguiente:

```
for (inicialización; condicion; incremento/decremento) {  
    //instrucción o bloque de instrucciones;  
}
```

Ejemplo:

```
document.write("Conteo hacia atrás<br>");  
for(var i=10; i>=0;i--){  
    document.write("<b>" + i + "</b>");  
    document.write("<br>");  
}  
document.write("Fin del conteo.");
```

El bucle *while*

Otra forma de crear un bucle es utilizando la sentencia *while*. Funciona de forma similar al *for*, pero su estructura sintáctica es completamente diferente. La estructura básica es:

```
while (condicion) {  
    //bloque de código;  
}
```

Donde, *condicion* es cualquier expresión JavaScript válida que se evalúe a un valor booleano. El bloque de código se ejecuta mientras que la condición sea verdadera. Por ejemplo:

Guía # 2: Estructuras de control: sentencias repetitivas y matrices

```
var nTotal = 35;
var con = 3;
while (con <= nTotal) {
    alert("El contador con es igual a: " + con);
    con++;
}
```

El resultado es el mismo que en el ciclo *for*, aunque su construcción sintáctica es muy diferente. El ciclo comprueba la expresión y continúa la ejecución del código mientras la evaluación sea true. El bucle *while* no tiene requiere que la variable de control del ciclo o lazo se modifique dentro del bloque de instrucciones para que en determinado momento se pueda detener la ejecución del lazo.

El bucle **do- while**

Esta instrucción es muy similar al ciclo *while*, con la diferencia que en esta, primero se ejecutan las instrucciones y luego, se verifica la condición. Esto significa que, el bloque de instrucciones se ejecutará con seguridad, al menos, una vez. Su sintaxis es:

```
do {
    //bloque de código ;
} while (expresión_condicional);
```

La diferencia con el anterior bucle estriba en que la *expresión condicional* se evalúa después de ejecutar el *bloque de código*, lo que garantiza que al menos una vez se ha de ejecutar, aún cuando la condición sea *false* desde el principio.

```
var nTotal = 35;
var con = 36;
do {
    alert("El contador con es igual a: " + con);
    con++;
} while (con <= nTotal);
```

Sentencias **break** y **continue**.

Para agregar aún más utilidad a los bucles, JavaScript incluye las sentencias **break** y **continue**. Estas sentencias se pueden utilizar para modificar el comportamiento de los ciclos más allá del resultado de su expresión condicional. Es decir, nos permite incluir otras expresiones condicionales dentro del bloque de código que se encuentra ejecutando el bucle.

El comando **break** causa una interrupción y salida del bucle en el punto donde se lo ejecute.

```
var nTotal = 35;
var con = 3;
do {
    if (con == 20)
        break;
    alert("El contador con es igual a: " + con);
    con++;
} while (con <= nTotal);
```

En este ejemplo, el bucle se repetirá hasta que la variable **con** llegue al valor 20, entonces se ejecuta la sentencia **break** que hace terminar el bucle en ese punto.

El comando **continue** es un tanto diferente. Se utiliza para saltar a la siguiente repetición del bloque de código, sin completar el pase actual por el bloque de comandos.

```
var nTotal = 70;
var con = 1;
do {
    if (con == 10 || con == 20 || con == 30 || con == 40)
        continue;
    alert("El contador con es igual a: " + con);
    con++;
} while (con <= nTotal);
```

En el ejemplo, cuando la variable **con** alcance los valores 10, 20, 30 y 40 **continue** salta el resto del bloque de código y comienza un nuevo ciclo sin ejecutar el método *alert()*.

Guía # 2: Estructuras de control: sentencias repetitivas y matrices

Sentencia *for-in*.

Este es un lazo o bucle relacionado con objetos y se utiliza para recorrer las diversas propiedades de un objeto. La sintaxis es la siguiente:

```
for (nombredevariable in objeto) {  
    //instruccion o bloque de instrucciones;  
}
```

Sentencia *with*

Sintaxis:

```
with(objeto){  
    instruccion o bloque de instrucciones;  
}
```

La instrucción *with* permite utilizar una notación abreviada al hacer referencia a los objetos. Es muy conveniente a la hora de escribir dentro del código del script un conjunto de instrucciones repetitivas relacionada con el mismo objeto. Por ejemplo, si se tiene el siguiente conjunto de instrucciones:

```
document.write("Hola desde JavaScript.");  
document.write("<br>");  
document.write("Estás aprendiendo el uso de una nueva instrucción de JavaScript.");
```

Podría escribirse abreviadamente utilizando el *with*, lo siguiente:

```
with(document){  
    write("Hola desde JavaScript.");  
    write("<br>");  
    write("Estás aprendiendo el uso de una nueva instrucción de JavaScript.");  
}
```

III. Materiales y Equipo.

Para la realización de la guía de práctica se requerirá lo siguiente:

No.	Requerimiento	Cantidad
1	Guía #2: Estructuras de control: sentencias repetitivas	1
2	Computadora con editor HTML/JavaScript y navegadores instalados	1

IV. Procedimiento.

Ejercicio #1: Encuesta que permite votar y seguir opinando sobre una pregunta. Cada vez que se emite la opinión se vuelve a preguntar si desea seguir votando. Mientras responda que si desea continuar, se le volverá a pedir la opinión, hasta que responda que ya no. En ese momento se emitirán los resultados. Se utiliza una técnica conocida como ciclo infinito para implementar la solución.

Guión 1: encuesta.html

```
<!DOCTYPE html>  
<html lang="es">  
<head>  
    <title>Encuesta sobre ley antimaras</title>  
    <meta charset="utf-8" />  
    <link  
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"  
rel="stylesheet">  
    <script  
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.  
js"></script>  
</head>  
<body>  
    <div id="app"></div>  
    <script src="js/encuesta.js" defer></script>  
</body>  
</html>
```

Guión 2: encuesta.js

```
'use strict';

// Declaración de variables (sin globales implícitas)
let voto;
let seguirVotando = true;
let cont1 = 0, cont2 = 0, cont3 = 0;
let total;
let per1, per2, per3;

const app = document.getElementById('app');

// Mostrar las instrucciones para responder
app.innerHTML = `
  <div class="container">
    <div class="row">
      <h1 class="text-center">
        Encuesta para determinar cuántas personas están a favor de la
        portabilidad numérica de teléfonos celulares.
      </h1>
    </div>
    <div class="row">
      <ul class="list-group list-group-flush">
        <li class="list-group-item fw-bold">Digite "1" si esta a
favor</li>
        <li class="list-group-item fw-bold">Digite "2" si esta en
contra</li>
        <li class="list-group-item fw-bold">Digite "3" si se abstiene
de opinar</li>
      </ul>
    </div>
  </div>
`;

// Ciclo repetitivo donde se captura voto por voto
// en tanto no se dé por terminado el ingreso de respuestas de la encuesta
while (seguirVotando) {
  voto = parseInt(prompt('¿Cuál es su voto?', ''), 10);

  switch (voto) {
    case 1:
      cont1++;
      break;
    case 2:
      cont2++;
      break;
    case 3:
      cont3++;
      break;
    default:
      alert('¡Voto no válido!');
  }

  // Se pregunta si se desea terminar la encuesta o continuar
  seguirVotando = confirm('¿Desea continuar s/n?');
}

// Obtener el total de respuestas de la encuesta
```

Guía # 2: Estructuras de control: sentencias repetitivas y matrices

```
total = cont1 + cont2 + cont3;

// Obtener los porcentajes (protegido contra división entre cero)
per1 = total > 0 ? Math.round((cont1 / total) * 10000) / 100 : 0;
per2 = total > 0 ? Math.round((cont2 / total) * 10000) / 100 : 0;
per3 = total > 0 ? Math.round((cont3 / total) * 10000) / 100 : 0;

// Mostrar los resultados de la encuesta (sin with, con template literal)
app.innerHTML += `
  <div class="container">
    <div class="row">
      <table class="table table-primary table-striped table-hover">
        <thead>
          <tr>
            <th>Resultado de los votos</th>
            <th>Votos obtenidos</th>
            <th>Porcentaje</th>
          </tr>
        </thead>
        <tbody>
          <tr>
            <td>Votos a favor</td>
            <td>${cont1}</td>
            <td>${per1} %</td>
          </tr>
          <tr>
            <td>Votos en contra</td>
            <td>${cont2}</td>
            <td>${per2} %</td>
          </tr>
          <tr>
            <td>Se abstienen de opinar</td>
            <td>${cont3}</td>
            <td>${per3} %</td>
          </tr>
        </tbody>
        <tfoot>
          <tr>
            <th>Totales</th>
            <th>${total}</th>
            <th>${(per1 + per2 + per3).toFixed(2)} %</th>
          </tr>
        </tfoot>
      </table>
    </div>
  </div>
`;
```

Indicaciones:

- Crear una carpeta con el nombre js y dentro de esta colocar los archivos encuesta.js, creado anteriormente.
- Crear una carpeta con el nombre css y dentro de esta colocar los archivos encuesta.css y zui-table.css, brindado en los recursos de la guía.

Resultado:

Esta página dice

¿Cuál es su voto?

Esta página dice

¿Desea continuar s/n?

Encuesta para determinar cuántas personas están a favor de la portabilidad numérica de teléfonoscelulares.

Digite "1" si esta a favor

Digite "2" si esta en contra

Digite "3" si se abstiene de opinar

Resultado de los votos	Votos obtenidos	Porcentaje
Votos a favor	6	67 %
Votos en contra	2	22 %
Se abstienen de opinar	1	11 %
Totales	9	100 %

Ejercicio #2: Creación de campos de formulario del tipo seleccionado en tiempo de ejecución con JavaScript.

Guión 1: forms.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <title>Creación de controles de formulario dinámicamente</title>
  <meta charset="utf-8" />
  <link
href="https://cdn.jsdelivrivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
  <script
src="https://cdn.jsdelivrivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.
js"></script>
  <script src="js/formcontrols.js" defer></script>
</head>
<body>
  <div class="container-fluid bg-info">
    <div class="row">
      <h1 class="display-1 text-center text-white">Creación de
formularios dinámicos</h1>
    </div>
  </div>
  <div class="container bg-light p-2">
    <div class="row position-relative">
      <form name="frmconf" id="form">
        <div class="input-group mb-3">
          <select class="form-select" name="selcontrol"
id="selcontrol">
            <option value="" selected>Tipo de control</option>
            <option value="text">Cuadro de texto</option>
            <option value="password">Cuadro de contraseña</option>
            <option value="textarea">Áreas de texto</option>
            <option value="checkbox">Casillas de
verificación</option>
            <option value="radio">Botones de opción</option>
            <option value="file">Control de archivo</option>
            <option value="button">Botones genéricos</option>
          </select>
        </div>
        <div class="input-group mb-3">
          <span class="input-group-text" id="basic-addon1">Número de
controles</span>
          <input type="text" name="txtnum" maxlength="2" autofocus
class="form-control">
        </div>
      </form>
    </div>
  </div>
```

Guía # 2: Estructuras de control: sentencias repetitivas y matrices

```
                placeholder="Número de controles">
            </div>
            <button class="btn btn-outline-primary" type="submit"
name="cmdenviar">Enviar</button>
        </form>
    </div>
</div>
<div class="container bg-light p-2">
    <div class="row" id="view"></div>
</div>
</body>
</html>
```

Guión 2: formcontrols.js

```
'use strict';

function init() {
    const form = document.getElementById('form');
    form.addEventListener('submit', (event) => {
        event.preventDefault();
        createForm(form.selcontrol.value, form.txtnum.value);
    });
}

function createForm(control, numero) {
    const cantidad = parseInt(numero, 10);
    const formView = document.getElementById('view');

    if (!control) {
        alert('No ha seleccionado el tipo de control');
        return;
    }

    if (!Number.isInteger(cantidad) || cantidad <= 0) {
        alert('Debe ingresar un número de controles válido');
        return;
    }

    let htmlForm = '<div class="row position-relative">';
    htmlForm += '<form name="miform">\n';
    htmlForm += '<h1 class="display-1 text-center">Formulario dinámico</h1>';

    for (let i = 0; i < cantidad; i++) {
        const id = `${control}${i + 1}`;

        switch (control) {
            case 'text':
            case 'password':
                htmlForm += `<input class="form-control" type="${control}"
name="${id}" required
                placeholder="Ingrese los datos en el campo ${control}"
/><br>\n`;
                break;

            case 'textarea':
                htmlForm += `<textarea class="form-control" name="${id}"
required
                placeholder="Ingrese los datos en el campo
${control}"></textarea><br />\n`;
        }
    }
}
```

```

        break;

        case 'checkbox':
            htmlForm += `<div>
                <input class="form-check-input" type="checkbox"
name="${id}" id="${id}" />
                <label class="form-check-label" for="${id}">${id}</label>
            </div>\n`;
            break;

        case 'radio':
            htmlForm += `<div>
                <label class="form-check-label" for="${id}">
                    <input class="form-check-input" type="radio"
name="${control}" id="${id}" />
                    <span>${id}</span>
                </label>
            </div>\n`;
            break;

        case 'file':
            htmlForm += `<label class="custom-file-input file-blue"><br />
                <input class="form-control" type="file" name="${id}" /><br
/>
            </label><br />\n`;
            break;

        case 'button':
            htmlForm += `<button class="btn btn-primary m-1"
name="${id}">${id}</button><br />\n`;
            break;
    }
}

htmlForm += `</form>\n</div>`;
formView.innerHTML = htmlForm;
}

window.addEventListener('load', init);

```

Indicaciones:

- Crear una carpeta con el nombre js y dentro de esta colocar los archivos with.js, creado anteriormente.
- Crear una carpeta con el nombre css y dentro de esta colocar los archivos basic.css y estilosform.css, brindado en los recursos de la guía.

Resultado en el navegador:

Creación de formularios dinámicos

Cuadro de texto

▼

Número de controles

4

Enviar

Formulario dinámico

Ingrese los datos en el campo text

Ingrese los datos en el campo text

Ingrese los datos en el campo text

Ingrese los datos en el campo text

Guión 1: math.html

Guión 2: with.js

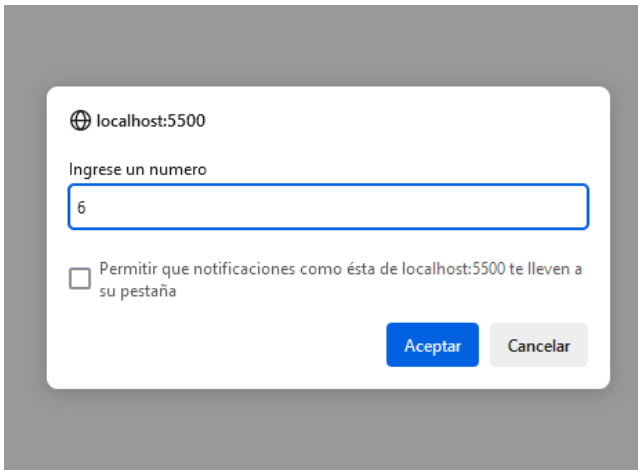
```
'use strict';

// Solicitar el valor para el radio del círculo
const radio = parseInt(prompt('Ingrese un número', ''), 10);

if (Number.isNaN(radio)) {
    alert('Debe ingresar un número válido');
} else {
    // Área de un círculo de radio "radio"
    const area = Math.PI * radio * radio;
    // Valor del lado horizontal definido por el radio
    const coorx = Math.abs(radio * Math.cos(Math.PI / 4));
    // Valor del lado vertical definido por el radio
    const coory = Math.abs(radio * Math.sin(Math.PI / 4));
    const pericir = 2 * Math.PI * radio;
    const perirec = 2 * coorx + 2 * coory;
```

Guía # 2: Estructuras de control: sentencias repetitivas y matrices

```
const resultados = document.getElementById('resultados');
resultados.innerHTML = `
    <p>El área del círculo es: ${area.toFixed(2)}</p>
    <p>El lado x del rectángulo generado es: ${coorx.toFixed(2)}</p>
    <p>El lado y del rectángulo generado es: ${coory.toFixed(2)}</p>
    <p>El perímetro del círculo es: ${pericir.toFixed(2)}</p>
    <p>El perímetro del rectángulo es: ${perirec.toFixed(2)}</p>
`;
}
```



localhost:5500

Ingrese un numero

6

☐ Permitir que notificaciones como ésta de localhost:5500 te lleven a su pestaña

Aceptar Cancelar

Resultados de los Cálculos

El área es: 113.10

El lado x del rectángulo generado es: 4.24

El lado y del rectángulo generado es: 4.24

El perímetro del círculo es: 37.70

El perímetro del rectángulo es: 16.97

V. Discusión de Resultados.

1. Cree un script que permita el ingreso de un número entero y muestre en pantalla la siguiente información:
 - 1) Cantidad de cifras, 2) Cantidad de cifras impares, 3) Cantidad de cifras pares, 4) Suma de cifras impares, 5) Suma de cifras pares, 6) Suma de todas las cifras, 7) Cifra mayor, 8) Cifra menor.
2. Crea un script que, al ingresar una palabra, realice los siguientes análisis: a) Cuente cuántas vocales hay en la palabra. b) Imprima la palabra al revés y determine si es un palíndromo.
3. Escribe un programa que usando bucles pueda calcular el factorial de un número dado.

VI. Investigación Complementaria.

1. Investigue para qué se utilizan las siguientes funciones del objeto Math: abs(), round(), ceil(), floor(), exp(), log() y random(). Ponga un ejemplo breve, de su utilización.

VII. Bibliografía.

- Flanagan, David. JavaScript La Guía Definitiva. 1ª Edición. Editorial ANAYA Multimedia / O'Reilly. 2007. Madrid, España.
- Terry McNavage. JavaScript Edición 2012. 1ª Edición. Editorial ANAYA Multimedia / Apress. Octubre 2011. Madrid, España.
- Tom Negrino / Dori Smith. JavaScript & AJAX Para Diseño Web. 6ª Edición. Editorial Pearson – Prentice Hall. 2007. Madrid España.

Guía # 2: Estructuras de control: sentencias repetitivas y matrices

- Powell, Thomas / Schneider, Fritz. JavaScript Manual de Referencia. 1ra Edición. Editorial McGraw-Hill. 2002. Madrid, España.
- McFedries, Paul. JavaScript Edición Especial. 1ra Edición. Editorial Prentice Hall. 2002. Madrid, España.